Assignment 02: Transformations, Viewing, and Projection

KESHAV SINGH, Indraprastha Institute of Information Technology, Delhi, India

This report describes camera motion along a 3D spiral for Assignment 02 of CSE 333/533. The camera rides on the spiral and looks at the origin, with perspective and orthographic projection, pause/resume and manual stepping. We derive the tangent vector analytically, build an orthonormal camera basis from it, and prove that this basis is never degenerate for the given spiral.

Additional Key Words and Phrases: spiral camera motion, viewing transformation, projection, orthonormal basis, OpenGL

ACM Reference Format:
Keshav Singh. 2026. Assignment 02: Transformations, Viewing, and Projection. 1, 1 (October 2026), 5 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

Author's Contact Information: Keshav Singh, keshav24300@iiitd.ac.in, Indraprastha Institute of Information Technology, Delhi, New Delhi, India.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM XXXX-XXXX/2026/10-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Notation and Design

The camera moves along the spiral

$$x(\theta) = r(\theta)\cos(k\theta), \tag{1}$$

$$y(\theta) = h\cos\theta, \tag{2}$$

$$z(\theta) = r(\theta)\sin(k\theta), \tag{3}$$

with $r(\theta) = \sqrt{R^2 + 4h^2\cos^2\theta}$, neck radius R, height $2h$, k cycles and $\theta \in [0, 2\pi]$; the program uses $R = h = 25$, $k = 8$. The program builds on the template code provided by the course teaching staff [**?**], written in C++ with OpenGL [**?**], GLFW [**?**] for the window and keyboard input, Dear ImGui [**?**] for the interface and GLM [**?**] for the view and projection matrices; the viewing and projection pipeline follows [LearnOpenGL].

Design choices. The camera state is only the scalar `spiralTheta` plus the flags `isPaused`, `isOrthographic` and `isq1`. The camera position and frame are recomputed from `spiralTheta` at every update, so no history is stored. Keys `1/2` select the Q1 camera (fixed Y up) or the Q2 camera (tangent-based frame).

2 Task 1

2.1 Camera on the spiral (Q1a)

`spiral()` evaluates eqs. (1) to (3) at `spiralTheta` to give the camera position $\mathbf{C}$, and

```
view = glm::lookAt(camPos, glm::vec3(0.0), glm::vec3(0.0, 1.0, 0.0));
```

looks at the origin with Y up. This is always valid: $\|\mathbf{C}\| \geq r(\theta) \geq R > 0$ and $\mathbf{C}$ has a nonzero horizontal part, so it is never parallel to the up vector. A frame is shown in fig. 1(a).

2.2 Projection toggle (Q1b)

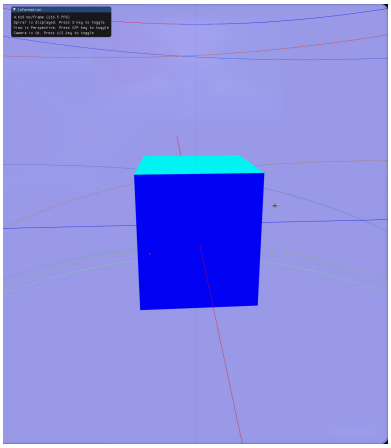
`O` sets `isOrthographic = true` and `P` sets it to false; the projection matrix is rebuilt immediately after each change. A frame is shown in fig. 1(b).

2.3 Pause/resume and manual stepping (Q1c)

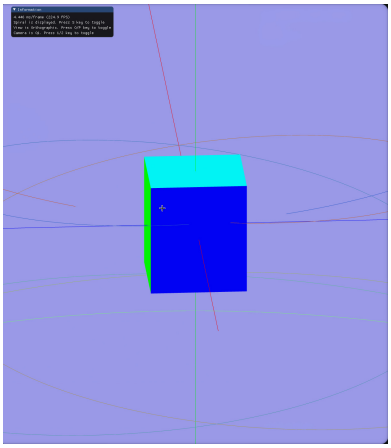
Space toggles `isPaused`; on resume the accumulator is cleared so the animation does not jump. While paused, Left/Right decrease/increase `spiralTheta` by half an animation step (0.125°) and update the view immediately. No wrapping is needed for manual steps since the curve is periodic.

```
if (!io.WantCaptureKeyboard && ImGui::IsKeyPressed(GLFW_KEY_SPACE)) {
  isPaused = !isPaused;
  if (!isPaused) timeAccumulator = 0;
}
if (!io.WantCaptureKeyboard && ImGui::IsKeyPressed(GLFW_KEY_LEFT) && isPaused) {
  spiralTheta -= spiralDeltaTheta * 0.5;
  setupViewTransformation(shaderProgram, spiralTheta);
}
if (!io.WantCaptureKeyboard && ImGui::IsKeyPressed(GLFW_KEY_RIGHT) && isPaused) {
  spiralTheta += spiralDeltaTheta * 0.5;
  setupViewTransformation(shaderProgram, spiralTheta);
}

float dt = ImGui::GetIO().DeltaTime;
```



(a) Perspective (Q1a)



(b) Orthographic (Q1b)

Fig. 1. Camera on the spiral looking at the origin with Y up.

```
timeAccumulator += dt;
if (timeAccumulator >= updateInterval && !isPaused) {
  timeAccumulator -= updateInterval;
  spiralTheta += spiralDeltaTheta;
  if (spiralTheta >= 2.0 * M_PI) spiralTheta -= 2.0 * M_PI;
  setupViewTransformation(shaderProgram, spiralTheta);
}
```

`O` / `P`	orthographic / perspective projection
`Space`	pause / resume
`Left` / `Right`	decrease / increase `spiralTheta` (paused)
`1` / `2`	Q1 camera / Q2 camera

3 Task 2

3.1 Tangent vector (Q2a)

We denote the tangent $\mathbf{v}(\theta)$ of the assignment by $\mathbf{t}(\theta)$, to keep $\mathbf{v}$ free for the camera basis below. From eq. (2), $dy/d\theta = -h\sin\theta$. Differentiating $r^2 = R^2 + 4h^2\cos^2\theta$ gives

$$r'(\theta) = -\frac{4h^2\cos\theta\sin\theta}{r(\theta)}. \tag{4}$$

By the product rule,

$$\frac{dx}{d\theta} = r'\cos k\theta - k\,r\sin k\theta, \tag{5}$$

$$\frac{dz}{d\theta} = r'\sin k\theta + k\,r\cos k\theta. \tag{6}$$

With $\mathbf{e}_r = (\cos k\theta, 0, \sin k\theta)$, $\mathbf{e}_\phi = (-\sin k\theta, 0, \cos k\theta)$, $\mathbf{e}_y = (0,1,0)$,

$$\mathbf{t}(\theta) = r'\,\mathbf{e}_r + k\,r\,\mathbf{e}_\phi - h\sin\theta\,\mathbf{e}_y. \tag{7}$$

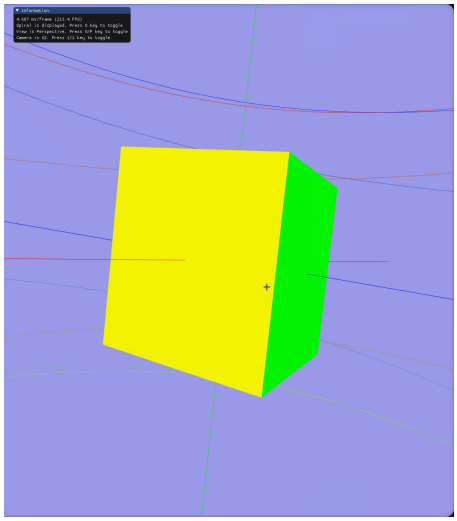


Fig. 2. Camera coordinate frame relative to the world frame and a small part of the spiral around the camera.

Since $kr \geq kR > 0$, $\mathbf{t} \neq \mathbf{0}$ for every θ. The program evaluates this in `spiral_derivative()`.

3.2 Orthonormal camera basis (Q2b)

Let $\mathbf{C}(\theta)$ be the camera position, $\mathbf{g} = -\mathbf{C}$ the gaze vector and $\mathbf{y} = (0, 1, 0)$. We build the basis $(\mathbf{u}, \mathbf{v}, \mathbf{w})$ (camera right, up and backward axes) as follows.

(1) $\mathbf{w} = -\mathbf{g}/\|\mathbf{g}\| = \mathbf{C}/\|\mathbf{C}\|$.
(2) $\mathbf{u}$ must lie in the plane of $\mathbf{g}$ and $\mathbf{t}$ and be perpendicular to $\mathbf{w}$, so $\mathbf{u} = \pm\mathbf{t}_\perp/\|\mathbf{t}_\perp\|$ with $\mathbf{t}_\perp = \mathbf{t} - (\mathbf{t}\cdot\mathbf{w})\mathbf{w}$.
(3) $\mathbf{v} = \mathbf{w} \times \mathbf{u}$ is then perpendicular to that plane, i.e. $\mathbf{v} = \pm\mathbf{n}/\|\mathbf{n}\|$ with $\mathbf{n} = \mathbf{t} \times \mathbf{C}$. The two signs are the only freedom left, and "$\mathbf{v}$ as close as possible to $\mathbf{y}$" selects the one with $\mathbf{v} \cdot \mathbf{y} > 0$.

Hence $\mathbf{v} = \mathbf{n}/\|\mathbf{n}\|$ (shown below to have $\mathbf{v} \cdot \mathbf{y} > 0$ always) and $\mathbf{u} = \mathbf{v} \times \mathbf{w} = -\mathbf{t}_\perp/\|\mathbf{t}_\perp\|$. In code, `updir = cross(-camPos, t)` $= \mathbf{n}$ is passed to `glm::lookAt(camPos, origin, updir)`, which builds exactly this frame (its right vector is $\mathbf{u}$ and its backward axis is $\mathbf{w}$). The schematic is in fig. 2.

Proposition 3.1. *For $R > 0$ and integer $k \geq 1$, $\mathbf{n} = \mathbf{t} \times \mathbf{C}$ is nonzero for every θ, $\{\mathbf{u}, \mathbf{v}, \mathbf{w}\}$ is an orthonormal basis, and $\mathbf{v} \cdot \mathbf{y} > 0$.*

Proof. Write $\mathbf{C} = r\,\mathbf{e}_r + y\,\mathbf{e}_y$ with $y = h\cos\theta$, and use eq. (7) with $y' = -h\sin\theta$. The cross products of the cylindrical basis are $\mathbf{e}_r \times \mathbf{e}_\phi = -\mathbf{e}_y$, $\mathbf{e}_r \times \mathbf{e}_y = \mathbf{e}_\phi$ and $\mathbf{e}_y \times \mathbf{e}_\phi = \mathbf{e}_r$. Expanding,

$$\mathbf{n} = -k\,r\,y\,\mathbf{e}_r - (r\,y' - y\,r')\,\mathbf{e}_\phi + k\,r^2\,\mathbf{e}_y. \tag{8}$$

The $\mathbf{e}_y$ component is $kr^2 \geq kR^2 > 0$, so $\|\mathbf{n}\| \geq kR^2 > 0$ for all θ; hence $\mathbf{C}$ and $\mathbf{t}$ are never parallel and $\mathbf{t}_\perp \neq \mathbf{0}$. Since $\mathbf{n} \perp \mathbf{C}$ and $\mathbf{n} \perp \mathbf{t}$, $\mathbf{v} = \mathbf{n}/\|\mathbf{n}\|$ is a unit vector orthogonal to $\mathbf{w}$ and $\mathbf{u}$, and $\mathbf{u} = \mathbf{v} \times \mathbf{w}$ completes a right-handed orthonormal basis. Finally $\mathbf{v} \cdot \mathbf{y} = kr^2/\|\mathbf{n}\| > 0$. □

3.3 Smoothness (Q2c)

By eq. (8), $\mathbf{n}$ is a smooth, 2π-periodic function of θ that never vanishes, so $\mathbf{v}$, $\mathbf{w}$ and $\mathbf{u}$ vary continuously, including at the loop closure $\theta = 2\pi \to 0$. Because $\mathbf{v} \cdot \mathbf{y} > 0$ for every θ, the sign choice is the same

everywhere and the camera stays upright, so no flipping is needed or performed. The animation is therefore smooth. The remaining visible effect is fast rotation about the Y axis, since the camera makes k turns around it per period, and the image is rolled relative to Task 1 because $\mathbf{v}$ is tied to the tangent rather than to $\mathbf{y}$.

Issue encountered. An earlier version of our code chose the sign of the up vector by testing the Y component of the camera's right vector, negating the up vector when it was positive. From eq. (8), that component is $u_y = hR^2 \sin\theta/(\|\mathbf{C}\|\|\mathbf{n}\|)$, which changes sign at $\theta = 0$ and $\theta = \pi$. The test therefore reversed the up vector at these two points: a 180° roll jump twice per loop (numerically 179.9° between samples 0.02° apart, versus an average of 0.88° between consecutive frames), with the camera upside-down for $\theta \in (0, \pi)$. The camera basis itself was never degenerate; only the sign normalisation was at fault. Testing $\mathbf{v} \cdot \mathbf{y}$ instead, or equivalently dropping the test (it never triggers by Proposition 3.1), removes the problem.

4 Animations

The recordings of the three animations (perspective, orthographic, and the tangent-based camera of Task 2) are included as `.mp4` files in the code folder of the submission.

References

[LearnOpenGL] *LearnOpenGL Shaders*. Accessed via https://learnopengl.com/Getting-started/Shaders.
[OpenGL] OpenGL: *Official Documentation*. Accessed via https://www.opengl.org/documentation/.
[Khronos] Khronos OpenGL Wiki: *Rendering Pipeline Overview*. Accessed via https://www.khronos.org/opengl/wiki/Rendering_Pipeline_Overview.
[GLFW] GLFW: *Documentation*. Accessed via https://www.glfw.org/documentation.html.
[docs.gl] docs.gl: *OpenGL API Reference*. Accessed via https://docs.gl.